

```
# Program 1: Compute GCD of two numbers
```

```
def compute_gcd(a, b):
```

```
    """Function to compute GCD using Euclid's algorithm"""
```

```
    while b != 0:
```

```
        a, b = b, a % b
```

```
    return a
```

```
# Input from user
```

```
try:
```

```
    num1 = int(input("Enter first number: "))
```

```
    num2 = int(input("Enter second number: "))
```

```
    if num1 == 0 and num2 == 0:
```

```
        print("GCD is undefined for both numbers zero.")
```

```
    else:
```

```
        gcd_value = compute_gcd(abs(num1), abs(num2))
```

```
        print(f"The GCD of {num1} and {num2} is: {gcd_value}")
```

```
except ValueError:
```

```
    print("Please enter valid integers only.")
```

```
PS C:\practical work> python -u "c:\practical work\python  
practical\p1.py"
```

Enter first number: 4

Enter second number: 32

The GCD of 4 and 32 is: 4

Program 2: Find Square Root using Newton's Method

```
def newton_sqrt(number, tolerance=1e-10):  
    """Calculate square root using Newton-Raphson  
    method."""  
    if number < 0:  
        raise ValueError("Cannot find square root of a negative  
        number.")  
    if number == 0:  
        return 0  
  
    guess = number / 2.0 # initial guess  
    while True:  
        next_guess = 0.5 * (guess + number / guess)
```

```
if abs(next_guess - guess) < tolerance:
```

```
    return next_guess
```

```
guess = next_guess
```

```
try:
```

```
    n = float(input("Enter a number to find its square root: "))
```

```
    result = newton_sqrt(n)
```

```
    print(f"The square root of {n} (approx.) is: {result:.6f}")
```

```
except ValueError as e:
```

```
    print(e)
```

```
PS C:\practical work> python -u "c:\practical work\python  
practical\p2.py"
```

```
Enter a number to find its square root: 441
```

```
The square root of 441.0 (approx.) is: 21.000000
```

```
# Program 3: Exponentiation (Power of a Number)
```

```
def power(base, exponent):
```

```
"""Compute base raised to the power exponent using  
fast exponentiation."""
```

```
result = 1
```

```
negative = False
```

```
if exponent < 0:
```

```
    exponent = -exponent
```

```
    negative = True
```

```
while exponent > 0:
```

```
    if exponent % 2 == 1: # if exponent is odd
```

```
        result *= base
```

```
    base *= base
```

```
    exponent //= 2
```

```
return 1 / result if negative else result
```

```
# Input section
```

```
try:
```

```
    base = float(input("Enter the base number: "))
```

```
    exp = int(input("Enter the exponent (integer): "))
```

```
print(f"{base} raised to the power {exp} is: {power(base,
exp)}")
```

```
except ValueError:
```

```
    print("Please enter valid numbers only.")
```

```
PS C:\practical work> python -u "c:\practical work\python
practical\p3.py"
```

```
Enter the base number: 2
```

```
Enter the exponent (integer): 6
```

```
2.0 raised to the power 6 is: 64.0
```

```
# Program 4: Find Maximum in a List of Numbers
```

```
def find_max(numbers):
```

```
    """Return the maximum value from a list of numbers."""
```

```
    if not numbers:
```

```
        raise ValueError("The list is empty, cannot find
maximum.")
```

```
    maximum = numbers[0]
```

```
for num in numbers[1:]:
```

```
    if num > maximum:
```

```
        maximum = num
```

```
return maximum
```

```
try:
```

```
    # Taking input as space-separated numbers
```

```
    nums = list(map(float, input("Enter numbers separated  
by spaces: ").split()))
```

```
    max_value = find_max(nums)
```

```
    print(f"The maximum number in the list is: {max_value}")
```

```
except ValueError as e:
```

```
    print(e)
```

```
PS C:\practical work> python -u "c:\practical work\python  
practical\p4.py"
```

```
Enter numbers separated by spaces: 2 3 44 43
```

```
The maximum number in the list is: 44.0
```

```
# Program 5: Linear Search and Binary Search
```

```
def linear_search(arr, target):  
    """Perform a linear search for target in arr."""  
    for i in range(len(arr)):  
        if arr[i] == target:  
            return i # return the index if found  
    return -1 # not found  
  
def binary_search(arr, target):  
    """Perform a binary search (works only for sorted  
lists)."""  
    low, high = 0, len(arr) - 1  
  
    while low <= high:  
        mid = (low + high) // 2  
        if arr[mid] == target:  
            return mid  
        elif arr[mid] < target:  
            low = mid + 1  
        else:  
            high = mid - 1
```

```
return -1
```

```
# --- Main Section ---
```

```
try:
```

```
    arr = list(map(int, input("Enter sorted list of numbers  
(space separated): ").split()))
```

```
    target = int(input("Enter the number to search: "))
```

```
    print("\n Linear Search:")
```

```
    linear_result = linear_search(arr, target)
```

```
    if linear_result != -1:
```

```
        print(f"Element found at position {linear_result + 1}")
```

```
    else:
```

```
        print("Element not found.")
```

```
    print("\n Binary Search:")
```

```
    binary_result = binary_search(arr, target)
```

```
    if binary_result != -1:
```

```
        print(f"Element found at position {binary_result + 1}")
```

```
    else:
```

```
        print("Element not found.")
```



```
except ValueError:
```

```
    print("Please enter valid integers only.")
```

```
PS C:\practical work> python -u "c:\practical work\python  
practical\p5.py"
```

```
Enter sorted list of numbers (space separated): 2 4 5 24 32
```

```
Enter the number to search: 5
```

```
Linear Search:
```

```
Element found at position 3
```

```
Binary Search:
```

```
Element found at position 3
```

```
# Program 6: Selection Sort and Insertion Sort
```

```
# --- Selection Sort ---
```

```
def selection_sort(arr):
```

```
    for i in range(len(arr)):
```

```
        min_index = i
```

```
for j in range(i + 1, len(arr)):
    if arr[j] < arr[min_index]:
        min_index = j
arr[i], arr[min_index] = arr[min_index], arr[i]
```

--- Insertion Sort ---

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
```

--- Main Program ---

```
nums = list(map(int, input("Enter numbers separated by
spaces: ").split()))
```

Copy list to use both sorts separately

```
arr1 = nums.copy()
```

```
arr2 = nums.copy()
```

```
selection_sort(arr1)
```

```
insertion_sort(arr2)
```

```
print("After Selection Sort:", arr1)
```

```
print("After Insertion Sort:", arr2)
```

```
PS C:\practical work> python -u "c:\practical work\python  
practical\p6.py"
```

```
Enter numbers separated by spaces: 3 62 22 37 9 44
```

```
After Selection Sort: [3, 9, 22, 37, 44, 62]
```

```
After Insertion Sort: [3, 9, 22, 37, 44, 62]
```

```
# Program 7: Merge Sort
```

```
def merge_sort(arr):
```

```
    if len(arr) > 1:
```

```
        mid = len(arr) // 2 # middle point
```

```
        left_half = arr[:mid]
```

```
        right_half = arr[mid:]
```

```
# Recursively sort both halves
```

```
merge_sort(left_half)
```

```
merge_sort(right_half)
```

```
# Merge the sorted halves
```

```
i = j = k = 0
```

```
while i < len(left_half) and j < len(right_half):
```

```
    if left_half[i] < right_half[j]:
```

```
        arr[k] = left_half[i]
```

```
        i += 1
```

```
    else:
```

```
        arr[k] = right_half[j]
```

```
        j += 1
```

```
    k += 1
```

```
# Copy any remaining elements
```

```
while i < len(left_half):
```

```
    arr[k] = left_half[i]
```

```
    i += 1
```

```
    k += 1
```

```
while j < len(right_half):
```

```
    arr[k] = right_half[j]
```

```
    j += 1
```

```
    k += 1
```

```
# --- Main Program ---
```

```
nums = list(map(int, input("Enter numbers separated by  
spaces: ").split()))
```

```
merge_sort(nums)
```

```
print("Sorted list after Merge Sort:", nums)
```

```
PS C:\practical work> python -u "c:\practical work\python  
practical\p7.py"
```

```
Enter numbers separated by spaces: 4 26 66 43 23 88
```

```
Sorted list after Merge Sort: [4, 23, 26, 43, 66, 88]
```

```
# Program 8: First n Prime Numbers
```

```
def is_prime(num):
```

```
    """Check if a number is prime."""
```

```
if num < 2:
    return False
for i in range(2, int(num**0.5) + 1):
    if num % i == 0:
        return False
return True
```

```
def first_n_primes(n):
    """Return a list of first n prime numbers."""
    primes = []
    num = 2
    while len(primes) < n:
        if is_prime(num):
            primes.append(num)
        num += 1
    return primes
```

```
# --- Main Program ---
```

```
try:
```

```
    n = int(input("Enter how many prime numbers you want:
"))
```

```
if n <= 0:
    print("Please enter a positive integer.")
else:
    print(f"First {n} prime numbers are:")
    print(first_n_primes(n))

except ValueError:
    print("Please enter a valid integer.")
```

```
PS C:\practical work> python -u "c:\practical work\python
practical\p8.py"
```

Enter how many prime numbers you want: 6

First 6 prime numbers are:

```
[2, 3, 5, 7, 11, 13]
```

Program 9: Matrix Multiplication

Function to multiply two matrices

```
def multiply_matrices(A, B):
```

```
    # Check if matrices can be multiplied
```

```
    if len(A[0]) != len(B):
```

```
    print("Matrix multiplication not possible! (Columns of  
A must equal rows of B)")
```

```
    return None
```

```
# Resultant matrix with zeros
```

```
result = [[0 for _ in range(len(B[0]))] for _ in range(len(A))]
```

```
# Multiplying matrices
```

```
for i in range(len(A)):
```

```
    for j in range(len(B[0])):
```

```
        for k in range(len(B)):
```

```
            result[i][j] += A[i][k] * B[k][j]
```

```
return result
```

```
# --- Main Program ---
```

```
# Example: You can change matrix size as needed
```

```
r1 = int(input("Enter number of rows for Matrix A: "))
```

```
c1 = int(input("Enter number of columns for Matrix A: "))
```

```
r2 = int(input("Enter number of rows for Matrix B: "))
```

```
c2 = int(input("Enter number of columns for Matrix B: "))
```



```
print("\nEnter elements of Matrix A:")
```

```
A = [[int(input(f"A[{i+1}][{j+1}]: ")) for j in range(c1)] for i in  
range(r1)]
```

```
print("\nEnter elements of Matrix B:")
```

```
B = [[int(input(f"B[{i+1}][{j+1}]: ")) for j in range(c2)] for i in  
range(r2)]
```

```
result = multiply_matrices(A, B)
```

```
if result:
```

```
    print("\nResultant Matrix (A × B):")
```

```
    for row in result:
```

```
        print(row)
```

```
PS C:\practical work> python -u "c:\practical work\python  
practical\p9.py"
```

```
Enter number of rows for Matrix A: 1
```

```
Enter number of columns for Matrix A: 2
```

```
Enter number of rows for Matrix B: 2
```

Enter number of columns for Matrix B: 1

Enter elements of Matrix A:

A[1][1]: 4

A[1][2]: 56

Enter elements of Matrix B:

B[1][1]: 88

B[2][1]: 23

Resultant Matrix ($A \times B$):

[1640]

Program 10: Word Count using Command Line Arguments

To run: python program10.py "your sentence here"

```
import sys
```

```
# Check if the user provided command line input
```

```
if len(sys.argv) < 2:
```

```
print("Usage: python program10.py <text>")
else:
    # Combine all arguments into one string
    text = " ".join(sys.argv[1:])
    words = text.split()
    print(f"Total words: {len(words)}")
    print("Words are:", words)
```

```
PS C:\practical work> python -u "c:\practical work\python
practical\p10.py"
```

```
Usage: python program10.py <text>
```

Program 11: Find Most Frequent Words in a Text File

```
def most_frequent_words(filename):
    """Reads a file and prints the most frequent words."""
    try:
        with open(filename, "r") as file:
            text = file.read().lower() # convert to lowercase for
            uniformity
```

```
# Remove punctuation and split words

for char in ",.!?;:-":
    text = text.replace(char, "")
words = text.split()

# Count word frequency manually
freq = {}
for word in words:
    freq[word] = freq.get(word, 0) + 1

# Find max frequency
max_count = max(freq.values())
most_common = [w for w, c in freq.items() if c ==
max_count]

print("\nMost frequent word(s):", ",
".join(most_common))
print("Frequency:", max_count)

except FileNotFoundError:
    print("File not found! Please check the filename.")
```

```
# --- Main Program ---
```

```
filename = input("Enter the filename (with extension): ")
```

```
most_frequent_words(filename)
```

```
PS C:\practical work> python -u "c:\practical work\python  
practical\p11.py"
```

```
Enter the filename (with extension): simple.txt
```

```
File not found! Please check the filename.
```

```
# Program 12: Simulate Elliptical Orbits in Pygame
```

```
import pygame, math, sys
```

```
# Initialize Pygame
```

```
pygame.init()
```

```
# Screen setup
```

```
WIDTH, HEIGHT = 800, 600
```

```
screen = pygame.display.set_mode((WIDTH, HEIGHT))
```

```
pygame.display.set_caption("Elliptical Orbit Simulation -  
Kunal Vats")
```

```
# Colors
```

```
BLACK = (0, 0, 0)
```

```
YELLOW = (255, 255, 0)
```

```
BLUE = (0, 150, 255)
```

```
# Orbit parameters
```

```
center_x, center_y = WIDTH // 2, HEIGHT // 2
```

```
a, b = 250, 150 # major and minor axes of ellipse
```

```
angle = 0
```

```
clock = pygame.time.Clock()
```

```
# Main loop
```

```
while True:
```

```
    for event in pygame.event.get():
```

```
        if event.type == pygame.QUIT:
```

```
            pygame.quit()
```

```
            sys.exit()
```

```
# Fill background
screen.fill(BLACK)

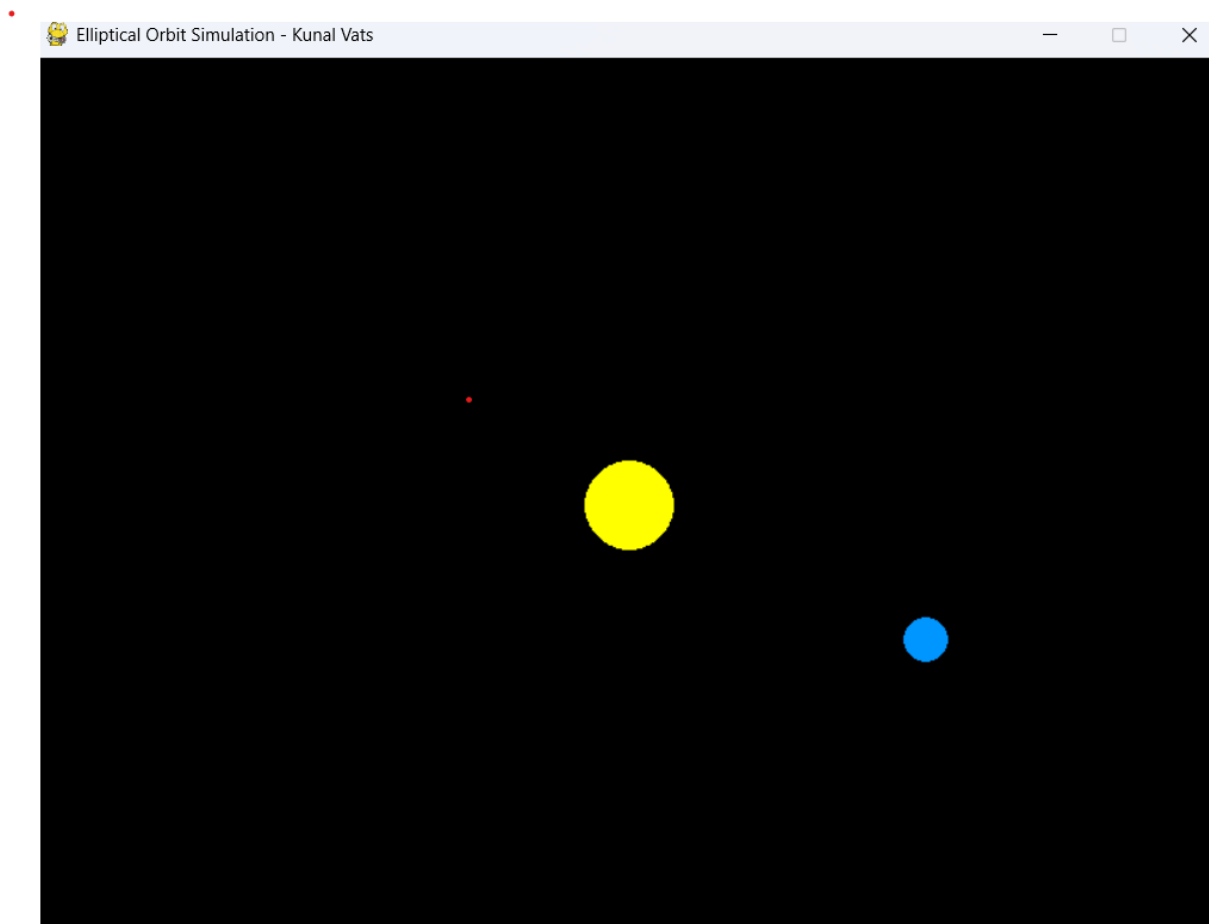
# Draw sun (center point)
pygame.draw.circle(screen, YELLOW, (center_x,
center_y), 30)

# Calculate planet position
x = center_x + int(a * math.cos(angle))
y = center_y + int(b * math.sin(angle))

# Draw planet
pygame.draw.circle(screen, BLUE, (x, y), 15)

# Update the angle for motion
angle += 0.01

# Refresh screen
pygame.display.flip()
clock.tick(60)
```



Program 13: Simulate Bouncing Ball using Pygame-

```
import pygame, sys
```

```
# Initialize pygame
```

```
pygame.init()
```



```
# Screen setup
```

```
WIDTH, HEIGHT = 800, 600
```

```
screen = pygame.display.set_mode((WIDTH, HEIGHT))
```

```
pygame.display.set_caption("Bouncing Ball Simulation -  
Kunal Vats")
```

```
# Colors
```

```
BLACK = (0, 0, 0)
```

```
RED = (255, 0, 0)
```

```
# Ball properties
```

```
ball_radius = 25
```

```
x, y = WIDTH // 2, HEIGHT // 2 # starting position
```

```
speed_x, speed_y = 5, 4 # ball speed in x and y
```

```
clock = pygame.time.Clock()
```

```
# Game loop
```

```
while True:
```

```
    for event in pygame.event.get():
```

```
        if event.type == pygame.QUIT:
```

```
pygame.quit()
```

```
sys.exit()
```

```
# Move the ball
```

```
x += speed_x
```

```
y += speed_y
```

```
# Bounce off the walls
```

```
if x - ball_radius <= 0 or x + ball_radius >= WIDTH:
```

```
    speed_x = -speed_x
```

```
if y - ball_radius <= 0 or y + ball_radius >= HEIGHT:
```

```
    speed_y = -speed_y
```

```
# Fill background
```

```
screen.fill(BLACK)
```

```
# Draw the ball
```

```
pygame.draw.circle(screen, RED, (x, y), ball_radius)
```

```
# Update screen
```

```
pygame.display.flip()
```

clock.tick(60)

